

EventHub - Phase Plan

Step-by-step phases: Creating, Backend, Frontend, Connecting, Security, Booking, Signature Features, Testing, Launch

Practice project | React + Spring Boot + MySQL

Companion to the EventHub Master Plan

Owner: Madhavan | 26 September 2026

Phase Overview

The project is built in **10 phases**. Each phase has one clear type of work, a list of steps, an output you can see, and a "done when" check. Do not start the next phase until the check passes.

Phase 0 FOUNDATIONS learn the basics	Phase 1 CREATING -> set up the project	Phase 2 BACKEND -> database + APIs	Phase 3 FRONTEND -> all pages + design	Phase 4 CONNECTING -> React talks to Spring
Phase 9 LAUNCH deploy + portfolio	Phase 8 TESTING <- TESTS + polish	Phase 7 SIGNATURE <- waitlist, QR, certs	Phase 6 BOOKING & PAYMENTS <- seat hold	Phase 5 SECURITY <- login, JWT, roles

Phase	Name	Main work	Time
0	Foundations	Java, JS and SQL basics + EventHub console app	2-3 weeks
1	Creating the project	Tools, folders, Spring Boot + React skeleton, Docker, Git, CI	1 week
2	Backend	Database tables + REST APIs for clubs, events, approval	2 weeks
3	Frontend	All pages and components with Tailwind, using sample data	2 weeks
4	Connecting	React uses the real APIs; search, filters, pagination	1 week
5	Security	Register, login, JWT cookie, roles, protected pages	1.5 weeks
6	Booking & Payments	Seat hold, no-oversell engine, My Tickets, Stripe	2 weeks
7	Signature features	Waitlist, QR check-in, certificates, feedback, analytics	2.5 weeks
8	Testing & polish	Automated tests, security checks, mobile and UI polish	1 week
9	Launch	Deploy, README, demo video, LinkedIn	1 week

Total: about 15-17 weeks at 1.5-2 hours per day (including Phase 0).

Phase 0 - Foundations

Time: 2-3 weeks

Goal: Learn the basics and build the EventHub rules as a simple Java console program.

Real-life example: Like learning to ride a bicycle in an empty ground before going on the road. The console app has the same rules as the real app, without the difficult parts.

Steps

1. Study the Learning Notes: Java basics, OOP, collections, exceptions
2. Create classes: Event, Student, Booking, and enum BookingStatus
3. Add menu options: 1. List events 2. Book 3. Cancel 4. Show waitlist
4. Throw a custom SoldOutException when seats are full
5. Use a Queue for the waitlist - cancelling gives the seat to the next student
6. JavaScript practice: a small HTML page that lists events from a JSON array with a search box
7. SQL practice: SELECT, WHERE, JOIN, GROUP BY on sample tables
8. Learn Git and push the console app to GitHub

Output of this phase

- EventHub console app (Java)
- Mini event list web page (HTML + JS)
- Your first GitHub repository

Done when

- Booking, sold out, cancel and waitlist all work in the console
- You can explain class vs object and List vs Queue

Skills you practise: OOP, collections, exceptions, enums, JavaScript arrays, SQL, Git

Phase 1 - Creating the Project

Time: 1 week

Goal: Create the real project structure so that the backend, frontend and database all run on your laptop.

Real-life example: Before building a house you prepare the land, bring the tools and draw the plan. This phase is exactly that.

Steps

1. Install JDK 21, IntelliJ IDEA, VS Code, Node.js (LTS), Docker Desktop, Postman, Git
2. Create the GitHub repository "eventhub" with backend/ and frontend/ folders
3. Generate the Spring Boot project from start.spring.io (Web, Data JPA, Validation, Security, MySQL, Flyway, Actuator, Mail, Cache, Lombok)
4. Create the React app with Vite; install Tailwind CSS, React Router, Redux Toolkit, Axios
5. Write docker-compose.yml with MySQL 8 and Mailpit (fake email inbox)
6. Connect Spring Boot to MySQL in application.yml; create dev and prod profiles
7. Add a /api/health endpoint and Swagger UI
8. Add GitHub Actions: build and test automatically on every push
9. Draw the page sketches and database diagram on paper (keep a photo in the repo)

Output of this phase

- Running Spring Boot app at localhost:8080
- Running React app at localhost:5173
- MySQL and Mailpit running in Docker
- Green CI tick on GitHub

Done when

- The React page shows "Backend: UP"
- docker compose up starts the database with one command

Skills you practise: Project setup, Maven, npm, Docker, configuration, CI/CD

Phase 2 - Backend

Time: 2 weeks

Goal: Build the database and the first REST APIs: clubs, events, tags and the approval workflow.

Real-life example: The kitchen is built before the dining hall is opened. Once the kitchen works, any menu (frontend) can use it.

Steps

1. Write Flyway migration V1: users, roles, profiles, clubs, club_members, events, tags, event_tags
2. Write Flyway migration V2: sample data - 5 clubs, 20 events, tags
3. Create entities with relationships (One-to-One, One-to-Many, Many-to-Many)
4. Create repositories with Spring Data JPA
5. Create DTOs (request / response) - never return entities directly
6. Create services with business rules
7. Event APIs: list with search, filter by tag / club / date, sort and pagination; get by id; create; update
8. Approval workflow: DRAFT -> PENDING_APPROVAL -> PUBLISHED / back to DRAFT with a reason
9. Validation (@Valid) and a global exception handler that returns clean JSON errors
10. Test every API in Postman and check Swagger

Output of this phase

- Database with 8+ tables and sample data
- Working REST APIs documented in Swagger
- Postman collection saved in the repo

Done when

- GET /api/events?tag=tech&page=0 returns the right events
- Submitting and approving an event changes its status correctly
- Wrong input returns a 400 error with a clear message

Skills you practise: Spring Boot, REST, JPA relationships, DTO pattern, validation, exception handling, Flyway

Phase 3 - Frontend

Time: 2 weeks

Goal: Build every page and component with a good design, using sample (fake) data first.

Real-life example: An architect first makes a model of the building. You can walk through every room and fix the design before the real building is connected to water and electricity.

Steps

1. Plan the design: colours, fonts, logo, dark mode (Tailwind theme)
2. Build shared components: Navbar, Footer, Button, Card, Modal, Badge, Loader, EmptyState
3. Set up React Router with all routes and a 404 page
4. Public pages: Home (upcoming + trending), Events with filter sidebar and pagination, Event Details, Club page
5. Auth pages: Login and Register with form validation
6. Student pages: Checkout with seat-hold countdown, My Tickets, Ticket with QR, Waitlist, Certificates
7. Organizer pages: My Club, Create / Edit Event form, Volunteers, Dashboard layout
8. Volunteer page: Gate Scanner screen (green / red result design)
9. Admin pages: Approval queue, Admin overview
10. Set up the Redux store with slices: auth, cart, notifications
11. Make every page mobile friendly

Output of this phase

- Complete clickable website with sample data
- Reusable component library
- Redux store ready for real data

Done when

- You can click through every page on laptop and phone
- Dark mode works on every page

Skills you practise: React components, props, state, hooks, React Router, Redux Toolkit, Tailwind, responsive design

Phase 4 - Connecting Frontend & Backend

Time: 1 week

Goal: Replace the sample data with real data from the Spring Boot APIs.

Real-life example: Now the model house gets real water and electricity - every tap and switch starts working.

Steps

1. Create one Axios client with the base URL and error handling
2. Enable CORS in Spring Boot for the React address
3. Load events, event details and clubs from the API
4. Connect search, filters, sorting and pagination to the API
5. Show loading skeletons, empty states and friendly error messages
6. Organizer create / edit event form saves to the backend
7. Admin approval queue approves and rejects for real

Output of this phase

- The website works with the real database

Done when

- An event added through the organizer form appears on the Events page after approval
- No sample data is left in the code

Skills you practise: Axios, async/await, useEffect, CORS, error handling

Phase 5 - Security

Time: 1.5 weeks

Goal: Real login and correct permissions for Student, Volunteer, Organizer and Admin.

Real-life example: The college gate: everyone shows an ID card, and each ID card opens only certain rooms.

Steps

1. Register API with BCrypt password hashing
2. Login API that sends a JWT inside an httpOnly cookie; logout clears it
3. JWT filter in Spring Security that reads the cookie on every request
4. CSRF protection with a CSRF cookie that Axios sends back
5. Roles: STUDENT and ADMIN; club roles: ORGANIZER and VOLUNTEER (per club)
6. Protect every API by role - for example only organizers of that club can edit its events
7. React: auth slice in Redux, protected routes, hide buttons the user cannot use
8. Audit log: record who approved, rejected or edited events
9. Rate-limit the login API to stop password guessing

Output of this phase

- Working register / login / logout
- Role-based access on backend and frontend

Done when

- A student gets 403 on admin APIs; an admin gets 200
- An organizer of Coding Club cannot edit Dance Club events

Skills you practise: Spring Security, JWT, cookies, CSRF, authentication vs authorization

Phase 6 - Booking & Payments

Time: 2 weeks

Goal: Students can book and pay safely - with no overselling, ever.

Real-life example: IRCTC tatkal booking - thousands of people click at the same moment, but a seat must never be given to two people.

Steps

1. Flyway V3: bookings and payments tables
2. Book API creates a HELD booking with a 10-minute timer and reduces seats
3. Optimistic locking with @Version on Event; retry when two requests clash
4. Scheduled job every minute: expire old holds and give the seats back
5. Free events confirm immediately
6. Stripe Checkout session for paid events (test mode)
7. Stripe webhook with signature check confirms the booking (Stripe CLI for local testing)
8. Idempotency - the same webhook twice never confirms twice
9. Cancel booking API; seats come back
10. Concurrency test: 100 parallel requests on 10 seats = exactly 10 bookings
11. React: Book button, countdown timer, Stripe redirect, success / cancel pages, My Tickets with status

Output of this phase

- Working booking engine with seat hold
- Stripe test payments
- Passing concurrency test

Done when

- Test card 4242 4242 4242 4242 confirms a booking
- An unpaid hold expires and the seat returns
- The concurrency test passes every time

Skills you practise: @Transactional, optimistic locking, @Scheduled, webhooks, idempotency, multithreading

Phase 7 - Signature Features

Time: 2.5 weeks

Goal: Add the features that make EventHub unique.

Real-life example: This is what turns a normal project into a project that interviewers remember.

Steps

1. Smart waitlist: join / leave, show position ("You are #3")
 - When a seat frees up, offer it to #1 for 30 minutes; if not accepted, move to the next
2. Notifications: emails with Spring Mail (see them in Mailpit) + in-app bell with unread count
 - Reminder email the day before the event (scheduled cron job)
3. QR e-tickets: generate a QR image (ZXing) holding a random ticket code
4. Gate scanner: volunteer scans with the phone camera (html5-qrcode); API answers VALID / ALREADY USED / INVALID
 - Live check-in counter for organizers
5. Certificates: after the event, checked-in students download a PDF (OpenPDF) with a unique number
 - Public verify page /verify/{number}
6. Feedback: 1-5 stars and a comment after the event
7. Analytics dashboard: bookings per day, revenue, check-in rate, rating (Recharts); cache results with @Cacheable

Output of this phase

- Waitlist, QR check-in, certificates, feedback and dashboards working

Done when

- Cancelling a booking sends an offer email to the first waitlisted student
- Scanning the same ticket twice shows ALREADY USED
- Only checked-in students get a certificate

Skills you practise: Queues, scheduling, email, QR and PDF generation, JPQL aggregates, caching, charts

Phase 8 - Testing & Polish

Time: 1 week

Goal: Make sure everything works, is secure and looks professional.

Real-life example: Before a car leaves the factory it goes through quality checks - brakes, lights, paint.

Steps

1. Unit tests for services with JUnit 5 and Mockito
2. Integration tests with Testcontainers (real MySQL in tests)
3. React Testing Library tests for key components (countdown, scanner result, forms)
4. Security checks: Actuator and Swagger only for ADMIN; no secrets in code
5. Accessibility and mobile check on every page
6. Fix bugs from a full manual run of the demo script
7. Clean up code, remove unused files, consistent naming

Output of this phase

- Automated test suite running in CI
- Bug-free demo journey

Done when

- CI is green with all tests
- Service test coverage is about 70% or more

Skills you practise: Testing pyramid, debugging, code quality

Phase 9 - Launch

Time: 1 week

Goal: Put EventHub online and present it to the world.

Real-life example: The grand opening of the restaurant - invite people, take photos, tell everyone.

Steps

1. Deploy the backend (Render or Railway) with a managed MySQL database
2. Deploy the frontend (Vercel or Netlify)
3. Set environment variables for secrets (JWT key, Stripe test keys, mail)
4. Write the README: problem, features, screenshots, tech stack, architecture diagram, how to run, live link
5. Record a 3-5 minute demo video following the demo script
6. Post on LinkedIn; add the project to your resume
7. Tag version v1.0 on GitHub

Output of this phase

- Live website link
- Professional README and demo video
- Resume and LinkedIn entries

Done when

- A friend can open the link on their phone, register, book, get a QR ticket and download a certificate

Skills you practise: Deployment, environment configuration, documentation, presenting your work

Phase Tracker

Fill in the dates and status as you go.

Phase	Name	Time	Start	Finish	Status
0	Foundations	2-3 wk			
1	Creating the project	1 wk			
2	Backend	2 wk			
3	Frontend	2 wk			
4	Connecting	1 wk			
5	Security	1.5 wk			
6	Booking & Payments	2 wk			
7	Signature features	2.5 wk			
8	Testing & polish	1 wk			
9	Launch	1 wk			